

Capstone Project Proposal and Execution Plan

The Tokenization Tax on Thinking: Latent vs. Explicit Test-Time Compute for Indic Reasoning Under Memory Constraints

Hardware: rented RTX 4090 (24 GB), sub-1B models only

PART I: THE PROJECT

1. Background

1.1 Why chain-of-thought is a bottleneck

Inside a transformer, the state of a reasoning step lives in the residual stream: a vector with thousands of dimensions that can hold several partial ideas, alternatives and degrees of uncertainty at once. Chain-of-thought (CoT) forces the model to collapse that state into a single discrete token at every step, and the next step is rebuilt from that token alone. It is like being made to think out loud one word at a time, where anything you did not say is forgotten. Whatever did not fit into the chosen token is lost.

1.2 Latent reasoning

Latent reasoning removes that bottleneck. In COCONUT (Hao et al., 2024), the model's last hidden state is fed straight back in as the next input embedding, a "continuous thought", with no projection onto the vocabulary. Training uses a curriculum: the model first learns ordinary CoT, then written reasoning steps are progressively replaced by continuous thoughts. Depth-recurrent models such as Huginn (Geiping et al., 2025) pursue the same idea by looping layers. An important finding is that continuous thoughts can encode several candidate next steps at once, so the model behaves more like a breadth-first search than a single committed path.

1.3 Multilingual models reason through English

Logit-lens studies of multilingual LLMs (e.g., Wendler et al., 2024) show a consistent three-phase pattern. Early layers map the input into a language-specific space, middle layers converge into a shared space whose states decode closest to English, and final layers map back into the output language. Low-resource languages converge less cleanly, and recent work finds that latent reasoning in large reasoning models degrades in such languages (Liu et al., 2026). Crucially, all of this evidence is observed through the vocabulary projection, by reading internal states as if they were tokens. Nobody has asked what happens to the English pivot in a model whose reasoning loop never touches the vocabulary.

1.4 Test-time scaling and the memory wall

Test-time scaling (TTS) improves reasoning without retraining by generating K candidate solution paths and selecting the best, through best-of- K sampling or tree search (Muennighoff et al., 2025; Hariri et al., 2026). Its cost is memory: every trajectory carries its own key-value (KV) cache, so memory grows with K multiplied by trace length. On consumer and edge GPUs this is the binding constraint (FastTTS, 2026). Two families of fixes exist: quantizing the KV cache to low precision (e.g., KIVI, Liu et al., 2024) and using a lightweight verifier on internal states to prune weak branches early (ReProbe, Ni et al., 2026). Both have been studied separately and almost entirely in English.

1.5 The link: the tokenization tax

Indic scripts typically take 2-4x more tokens than English for the same content under common tokenizers. Explicit reasoning memory scales with token count, and TTS multiplies it by K , so Indic reasoning hits the memory wall 2-4x sooner: a double tax of longer traces times more trajectories. Latent reasoning is tokenizer-agnostic during its thinking phase. Ten continuous thoughts cost the same in Telugu as in English, and because continuous thoughts can hold multiple candidate steps, latent reasoning may act as a form of parallel search without K separate caches. If latent reasoning works in Indic languages, its memory advantage over explicit search should be largest exactly where current models are weakest. This observation is what joins the two halves of the project.

2. Research questions

Main question: Is latent reasoning a memory-efficient substitute for explicit test-time search, and does its advantage grow for languages that tokenize badly?

RQ1 (the pivot): When the vocabulary projection is removed from the reasoning loop, does the English pivot survive or dissolve? If it survives, latent reasoning is compressed English thinking with the English hidden. If it dissolves, latent reasoning is structurally different from CoT and offers a route to closing the Indic reasoning gap.

RQ2 (the tax): How much does tokenization inflate the memory cost of explicit TTS in Indic languages, and how much of that cost can KV quantization and probe-guided pruning recover on a 24 GB GPU?

RQ3 (the bridge): At matched peak memory, how does latent reasoning compare with explicit search across languages? How robust are continuous thoughts to quantization, given that a quantized latent thought is a corrupted input to the next step rather than just a corrupted cache? Can an internal-state probe decide how many latent steps to take?

Every question is informative in either direction: convergence and non-convergence, large or small tax, latent advantage or no advantage are all findings.

3. Project structure: three pillars

Pillar	Question	Stands alone as
A. English pivot in latent models (RQ1)	When the vocabulary projection is removed from the reasoning loop (COCONUT), does the English pivot seen in multilingual LLMs survive or dissolve?	Interpretability paper
B. Memory-constrained explicit TTS (RQ2)	How far do KV-cache quantization and probe-guided branch pruning extend TTS on a 24 GB card, and how large is the tokenization tax across languages?	Workshop paper; needs no Indic training data
C. Latent vs. explicit at matched memory (RQ3)	Accuracy vs. peak memory for both approaches across languages; robustness of latent thoughts to quantization; probe-guided adaptive latent depth.	Main paper contribution

Each pillar can be published on its own, so a failure in one does not sink the project. Pillars A and B share tooling: the probes used to read latent representations in A are the same machinery used as a pruning verifier in B.

4. Novelty and publishability

An empty intersection. Existing multilingual latent-reasoning work probes emergent latent computation in ordinary models. To our knowledge, no work trains an architecturally latent model multilingually, and none measures how tokenization inflates the memory cost of test-time scaling across languages.

Open questions, not incremental gains. The project does not target a benchmark improvement of a few points. It asks whether a known property of multilingual models (the English pivot) transfers to a new class of architectures, and whether latent reasoning changes the cost structure of test-time compute.

Stated gaps in recent literature. Recent papers call for necessity-first evaluation of latent reasoning (Dilgren & Wiegrefe, 2026; Aswal et al., 2026), and TTS surveys identify memory as the main deployment barrier (Hariri et al., 2026; FastTTS, 2026).

Venues. The Workshop on Latent & Implicit Thinking Beyond CoT Reasoning and efficiency-focused workshops for early results; ACL or EMNLP for the full paper.

Scoop resistance. The full combination spans three research communities (multilingual NLP, latent reasoning, efficient inference), which makes an identical project elsewhere unlikely over the project horizon.

5. Deliverables

The project will produce: translated and answer-verified reasoning data for Hindi, Telugu and a fourth Indic language; an evaluation protocol that reports accuracy against peak memory across languages; tokenization-tax measurements across tokenizers; trained latent-reasoning checkpoints with matched CoT baselines and controls; an analysis of latent representations through probing and causal tracing; open-source code; a workshop paper after the first semester and a full paper at the end.

PART II: THE EXECUTION PLAN

6. Models

Primary: Qwen3-0.6B-Base. Multilingual and strong for its size. A full fine-tune fits on a 4090 (about 10 GB for weights, gradients and optimizer state, plus activations) with gradient checkpointing. The Base version is used because it is the cleaner starting point for COCONUT training and probing.

Debug model: Gemma 3 270M. Used to iterate the pipeline cheaply. Its large vocabulary should tokenize Indic scripts better. Not used for headline results.

Tokenizer-only comparison: Sarvam-1. Its Indic-optimized tokenizer is loaded without model weights, to separate "Indic languages are harder" from "Indic languages are tokenized worse".

Qwen3-0.6B has a large KV cache for its size (28 layers, 8 KV heads, head dim 128; about 112 KB per token in fp16). This makes the memory wall appear naturally on a 24 GB card:

Configuration	Tokens in cache	KV cache (fp16)	Fits on 4090?
K = 32, 4k-token traces	~131k	~15 GB	Yes
K = 64, 4k-token traces	~262k	~30 GB	No
K = 32, Telugu traces at ~2.5x length	~328k	~37 GB	No

7. Phase 0: go/no-go checks (first 3-4 weeks)

Check	Cost	Decision it informs
Tokenizer fertility (tokens per word) for Qwen3, Gemma 3 and Sarvam-1 on English, Hindi, Telugu	CPU only	First tokenization-tax result; sets sequence-length budgets for training.
Logit-lens pivot check on Qwen3-0.6B-Base with Hindi and Telugu inputs	A few GPU-hours	Most important gate. Pivot findings come from 7B-class models; if a 0.6B model shows no pivot, Pillar A is reframed (see risks).
Baseline MGSM accuracy per language	A few GPU-hours	Floor for TTS gains. If near zero in Telugu, add an easier task tier.
Reproduce official COCONUT repo on GPT-2, English GSM8K	~10 GPU-hours	Pipeline sanity before any Indic run.
IndicTrans2 translation on a small sample	A few GPU-hours	Translation quality and filtering rules.

8. Data and evaluation

Languages: English, Hindi, Telugu (Dravidian, for typological distance), plus one flexible fourth language. Bengali is convenient because MGSM covers it, though it is linguistically close to Hindi. If distance matters more than evaluation convenience, a second Dravidian language is preferred.

Training data: a 40k-problem subset of GSM8K-style augmented reasoning traces, translated into each Indic language with IndicTrans2 (AI4Bharat) on a 4090. Examples where translation altered numbers or the final answer are filtered out.

Evaluation: MGSM for Telugu and Bengali (250 problems per language, evaluation only), plus a held-out translated GSM8K test split for the remaining languages.

9. Phased timeline

Phase	Work	Deliverable
Phase 0 (3-4 weeks)	Checks in Section 7; translation pipeline set up.	Go/no-go decision; fertility measurements.
Semester 1: Pillar B + data	TTS sweeps with vLLM or llama.cpp over K and KV precision (fp16 / 8 / 4 / 2-bit) across languages, using KIVI or llama.cpp KV quant types rather than custom kernels. Train the ReProbe-style pruning probe. Finish translating training data.	Workshop paper: tokenization tax + memory-constrained TTS (the safety net).
Semester 2: Pillar A	Stage gate first: English + Telugu, one seed. If the signal holds, full run grid (Section 10). Probing and causal tracing.	Latent checkpoints; pivot analysis.
Semester 3: Pillar C + writing	Latent vs. explicit at matched memory; quantization robustness of latent thoughts; probe-guided adaptive latent depth. Mostly inference on existing checkpoints and the Semester 1 harness.	Full paper (ACL/EMNLP target) + released benchmark and code.

10. Pillar A run design

COCONUT's curriculum begins with a stage of ordinary CoT fine-tuning, so each latent run's stage-0 checkpoint serves as the CoT baseline. Controls branch from that checkpoint instead of training from scratch, and are run only on the contrast that matters.

Configuration	Languages	Seeds	Cost (run-equivalents)
Latent run (stage-0 checkpoint = CoT baseline)	4	3	12
Recurrence-removed control, branched from stage 0	2 (EN, TE)	1	~1.4
Curriculum-perturbed control, branched from stage 0	2 (EN, TE)	1	~1.4
Total			~15

If a reviewer requires a step-matched CoT baseline, extend the stage-0 checkpoint on CoT data for the final seeds only.

11. Compute budget (RTX 4090 hours)

Workstream	4090-hours
Phase 0: checks, GPT-2 COCONUT reproduction, translation setup	~30-40
Pillar B: TTS sweeps, probes, integration testing	~100-150
Translation of training data	~15-25
Pillar A: ~15 run-equivalents + probing and causal tracing	~150-250
Pillar C: mostly inference on existing checkpoints	~50-100
Total	~350-550
Total with 30% buffer for failed runs	~450-700

Assumptions: ~40k examples per language at ~250 tokens (~10M tokens per epoch); ~20 epochs across the curriculum; 20-30% GPU utilization during COCONUT's sequential latent passes; one full latent run per language at ~8-15 hours. Treat all figures as $\pm 50\%$ until Phase 0 measures real throughput. Multiply by the college's hourly rental rate for the cost.

12. Risks and fallbacks

Risk	Mitigation
Qwen3-0.6B shows no English pivot	Reframe Pillar A as whether latent training creates a shared cross-lingual space; lean on Pillars B and C for the main result.
Translated reasoning data too noisy	Stricter answer-consistency filtering; drop to three languages; check AI4Bharat releases for native data.
KV quantization gains unremarkable (crowded area; KIVI is direct prior art and must be cited)	Pillar B's contribution is the multilingual tokenization-tax measurement, not a new quantizer.
0.6B reasoning too weak for TTS to help in Telugu	Add an easier or synthetic task tier where base accuracy is non-trivial.
One pillar gets scooped	Remaining pillars and the bridge question still stand.
Rented sessions interrupted	Checkpoint every 30-60 minutes; all runs are single-GPU and resume trivially.

13. Practical notes for rented 4090s

Use bf16, FlashAttention 2 (supported on the 4090's Ada architecture) and 8-bit AdamW to leave headroom for longer Telugu sequences. Run seeds in parallel on separate cards rather than parallelizing a single run; with 3-4 cards at once, Semester 2 training finishes in a couple of weeks of wall-clock time. Measure the tokenization tax on real data in Phase 0 before fixing maximum sequence lengths.

References

Aswal et al. (2026). Observable Patterns Are Not Explanations. arXiv:2606.12689
Dilgren & Wiegrefe (2026). Are Latent Reasoning Models Easily Interpretable? arXiv:2604.04902
FastTTS (2026). Accelerating Test-Time Scaling for Edge LLM Reasoning. ASPLOS 2026. arXiv:2509.00195
Gala et al. (2023). IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages. arXiv:2305.16307
Geiping et al. (2025). Scaling Test-Time Compute with Latent Reasoning (Huginn). arXiv:2502.05171
Hao et al. (2024). Training LLMs to Reason in a Continuous Latent Space (COCONUT). arXiv:2412.06769
Hariri et al. (2026). Test-Time Scaling in Reasoning LLMs. arXiv:2608.04001
Liu et al. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750
Liu et al. (2026). Large Reasoning Models Are (Not Yet) Multilingual Latent Reasoners. arXiv:2601.02996
Muennighoff et al. (2025). s1: Simple Test-Time Scaling. arXiv:2501.19393
Ni et al. (2026). ReProbe: Efficient Test-Time Scaling of Multi-Step Reasoning by Probing Internal States. ACL 2026. arXiv:2511.06209
Shi et al. (2022). Language Models are Multilingual Chain-of-Thought Reasoners (MGSM). arXiv:2210.03057
Tezuka et al. (2025). The Transfer Neurons Hypothesis. arXiv:2509.17030
Wendler et al. (2024). Do Llamas Work in English? On the Latent Language of Multilingual Transformers. arXiv:2402.10588